

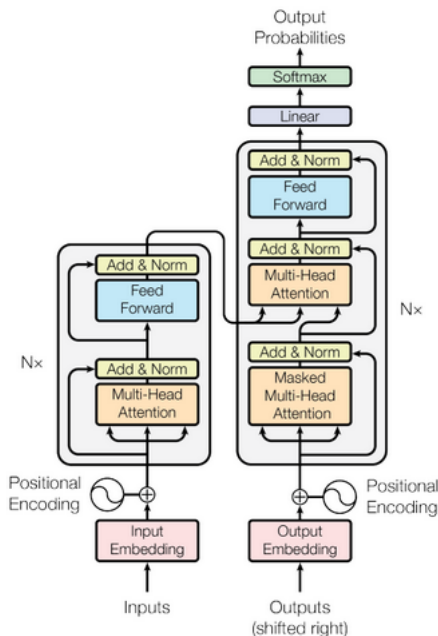
Classifier transfer learning from LLM 情緒類別分類器(遷移學習)

This is a comprehensive notebook and tutorial on how to fine tune the `qwen-0.5b` classification model

Fine-tuning Qwen-0.5B (a smaller model) with LoRA (Low-Rank Adaptation) is an efficient approach that requires less computational power.

It got a classification accuracy of 0.93.

Transformers 變形金剛(變壓器)模型



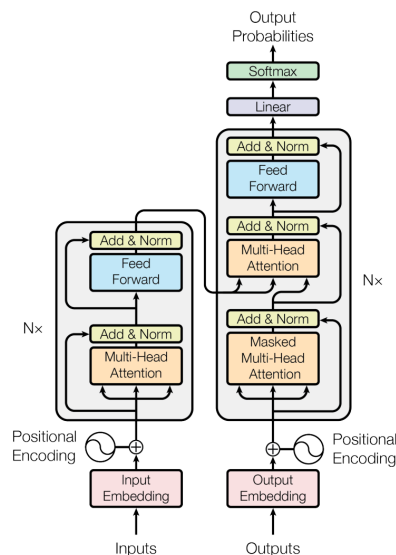
Bert and GPT are parts of Transformer

BERT

Encoder

GPT

Decoder



Encoders and Decoders As mentioned, there are encoders and decoders. BERT uses encoders only, GTP uses decoders only. Both options understand language including syntax and semantics. Especially the next generation of large language models like GPT with billions of parameters do this very well.

The two models focus on different scenarios. However, since the field of foundation models is evolving, the differentiation is often fuzzier.

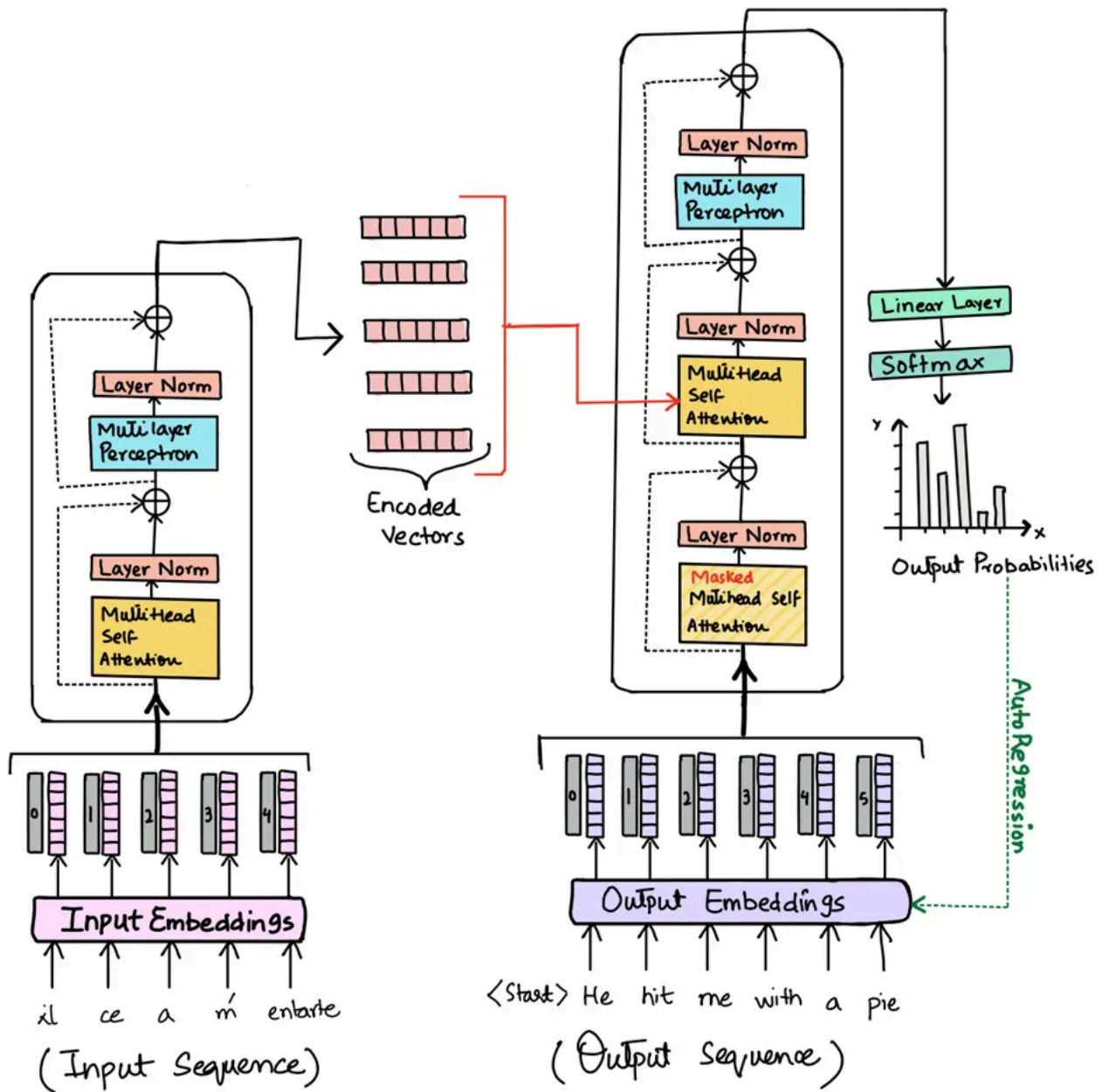
BERT (encoder): classification (e.g., sentiment), questions and answers, summarization, named entity recognition GPT (decoder): translation, generation (e.g., stories) The outputs of the core models are different:

BERT (encoder): Embeddings representing words with attention information in a certain context GPT (decoder): Next words with probabilities Both models are pretrained and can be reused without intensive training. Some of them are available as open source and can be downloaded from communities like Hugging Face, others are commercial. Reuse is important, since trainings are often very resource intensive and expensive which few companies can afford.

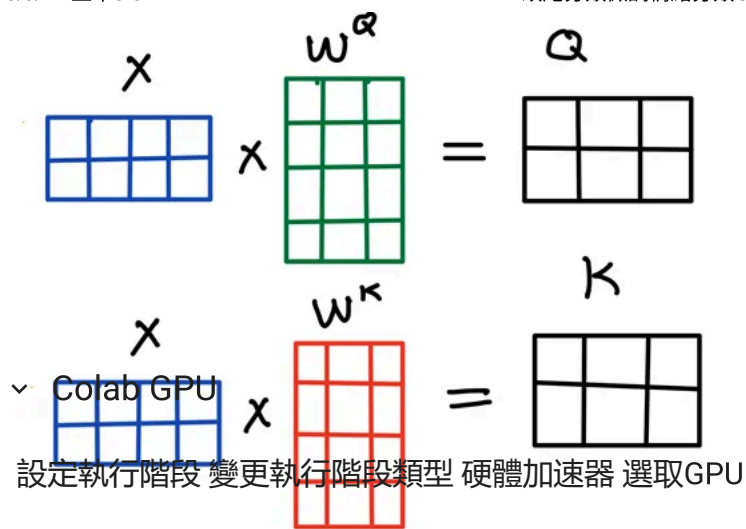
The pretrained models can be extended and customized for different domains and specific tasks. Layers can sometimes be reused without modifications and more layers are added on top. If layers need to be modified, the new training is more expensive. The technique to customize these models is called Transfer Learning, since the same generic model can easily be transferred to other domains.

[Source](#)

Encoder-Decoder Transformers



Attention Layer



```
!nvidia-smi
```

Mon May 26 14:35:21 2025

NVIDIA-SMI 550.54.15		Driver Version: 550.54.15		CUDA Version: 12.4	
GPU	Name	Persistence-M	Bus-Id	Disp.A	Volatile Uncorr. ECC
Fan	Temp	Perf	Pwr:Usage/Cap	Memory-Usage	GPU-Util Compute M. MIG M.
0	Tesla T4	Off	00000000:00:04:0	Off	0
N/A	53C P8	10W / 70W	0MiB / 15360MiB	0%	Default N/A

Processes:							GPU Memory Usage
GPU ID	GI ID	PID	Type	Process name			
No running processes found							

```
# 列出GPU與CPU資源
```

```
from tensorflow.python.client import device_lib
print(device_lib.list_local_devices())
```

```
[name: "/device:CPU:0"
device_type: "CPU"
memory_limit: 268435456
locality {
}
incarnation: 14482523020291324203
xla_global_id: -1
, name: "/device:GPU:0"
device_type: "GPU"
memory_limit: 14619377664
locality {
  bus_id: 1
  links {
  }
}
incarnation: 2049058413601720184
physical_device_desc: "device: 0, name: Tesla T4, pci bus id: 0000:00:04.0, compute capability: 7.5"
xla_global_id: 416903419
]
```

```
# Specify the GPU device to use
```

```
#import os
```

```
#os.environ["CUDA_VISIBLE_DEVICES"] = "0"
```

✓ Ubuntu版本資訊

```
!lsb_release -a
```

```
No LSB modules are available.
Distributor ID: Ubuntu
Description: Ubuntu 22.04.4 LTS
Release: 22.04
Codename: jammy
```

✓ Insatll packages

```
transformers: For model handling.
peft: For LoRA integration.
datasets: For dataset handling.
accelerate: For distributed training.
bitsandbytes: For memory-efficient training (optional but useful for Qwen).
```

```
#!pip install transformers peft datasets accelerate bitsandbytes
```

```
!pip install evaluate
```

```
Collecting evaluate
  Downloading evaluate-0.4.3-py3-none-any.whl.metadata (9.2 kB)
Requirement already satisfied: datasets>=2.0.0 in /usr/local/lib/python3.11/dist-packages (from evaluate) (2.14.4)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.11/dist-packages (from evaluate) (2.0.2)
Requirement already satisfied: dill in /usr/local/lib/python3.11/dist-packages (from evaluate) (0.3.7)
Requirement already satisfied: pandas in /usr/local/lib/python3.11/dist-packages (from evaluate) (2.2.2)
Requirement already satisfied: requests>=2.19.0 in /usr/local/lib/python3.11/dist-packages (from evaluate) (2.32.3)
Requirement already satisfied: tqdm>=4.62.1 in /usr/local/lib/python3.11/dist-packages (from evaluate) (4.67.1)
Requirement already satisfied: xxhash in /usr/local/lib/python3.11/dist-packages (from evaluate) (3.5.0)
Requirement already satisfied: multiprocess in /usr/local/lib/python3.11/dist-packages (from evaluate) (0.70.15)
Requirement already satisfied: fsspec>=2021.05.0 in /usr/local/lib/python3.11/dist-packages (from fsspec[http]>=2021.05.0->evaluate) (2025.3.2)
Requirement already satisfied: huggingface-hub>=0.7.0 in /usr/local/lib/python3.11/dist-packages (from evaluate) (0.31.2)
Requirement already satisfied: packaging in /usr/local/lib/python3.11/dist-packages (from evaluate) (24.2)
Requirement already satisfied: pyarrow>=8.0.0 in /usr/local/lib/python3.11/dist-packages (from datasets>=2.0.0->evaluate) (18.1.0)
Requirement already satisfied: aiohttp in /usr/local/lib/python3.11/dist-packages (from datasets>=2.0.0->evaluate) (3.11.15)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.11/dist-packages (from datasets>=2.0.0->evaluate) (6.0.2)
Requirement already satisfied: filelock in /usr/local/lib/python3.11/dist-packages (from huggingface-hub>=0.7.0->evaluate) (3.18.0)
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.11/dist-packages (from huggingface-hub>=0.7.0->evaluate) (4.12.2)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from requests>=2.19.0->evaluate) (3.4.2)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from requests>=2.19.0->evaluate) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from requests>=2.19.0->evaluate) (2.4.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packages (from requests>=2.19.0->evaluate) (2025.4.26)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.11/dist-packages (from pandas->evaluate) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-packages (from pandas->evaluate) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dist-packages (from pandas->evaluate) (2025.2)
Requirement already satisfied: aiohappyeyeballs>=2.3.0 in /usr/local/lib/python3.11/dist-packages (from aiohttp->datasets>=2.0.0->evaluate) (2.6.0)
Requirement already satisfied: aiosignal>=1.1.2 in /usr/local/lib/python3.11/dist-packages (from aiohttp->datasets>=2.0.0->evaluate) (1.3.2)
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.11/dist-packages (from aiohttp->datasets>=2.0.0->evaluate) (25.3.0)
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.11/dist-packages (from aiohttp->datasets>=2.0.0->evaluate) (1.6.0)
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.11/dist-packages (from aiohttp->datasets>=2.0.0->evaluate) (6.4.3)
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.11/dist-packages (from aiohttp->datasets>=2.0.0->evaluate) (0.3.1)
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.11/dist-packages (from aiohttp->datasets>=2.0.0->evaluate) (1.20.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packages (from python-dateutil>=2.8.2->pandas->evaluate) (1.17.0)
Downloading evaluate-0.4.3-py3-none-any.whl (84 kB)
84.0/84.0 kB 2.6 MB/s eta 0:00:00

Installing collected packages: evaluate
Successfully installed evaluate-0.4.3
```

✓ 掛載雲端硬碟

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

✓ 切換工作目錄到雲端硬碟目錄下(取用資料比較方便，可以用相對路徑)

你要改成你自己雲端硬碟目錄的路徑，可複製路徑並貼上，不要用鍵盤輸入！

```
# cd to your folder
# cd命令的前面一行不要加上說明文字，否則colab的cd會認不得指令
```

```
cd /content/drive/MyDrive/bigdata
```

```
/content/drive/MyDrive/bigdata
```

```
ls -l
```

```
total 13818
-rw-r--r-- 1 root root 73148 May 26 14:37 1-改尾分類微調情緒分類Qwen0.5b-v4-5epochs-acc0.93.ipynb
```

```
-rw----- 1 root root      4857 May 26 14:33 custom_qwen_model.py
-rw----- 1 root root 14071136 May 26 14:33 dataset_reviews.csv

import torch
import datasets
import pandas as pd
import evaluate
import numpy as np

# Load Huggingface transformers
from transformers import AutoTokenizer, AutoModelForSequenceClassification, AutoModelForCausalLM
from transformers import TrainingArguments, Trainer
from transformers import BertTokenizer, BertTokenizerFast, BertForSequenceClassification
from sklearn import metrics
import torch
import evaluate
import datasets

# Setting device on GPU if available, else CPU
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('Using device:', device)
```

↗ Using device: cuda

Read data

簡體中文資料，資料集來自於網路，轉換成繁體中文

```
df = pd.read_csv('./dataset_reviews.csv', sep='|')
```

df



	text	label
0	做父母一定要有劉墉這樣的心態，不斷地學習，不斷地進步，不斷地給自己補充新鮮血液，讓自己保持一...	1
1	作者真有英國人嚴謹的風格，提出觀點、進行論述論證，儘管本人對物理學瞭解不深，但是仍然能感受到...	1
2	作者長篇大論借用詳細報告數據處理工作和計算結果支持其新觀點。為什麼荷蘭曾經縣有歐洲最高的生產...	1
3	作者在戰幾時之前用了『擁抱』令人叫絕，日本如果沒有戰敗，就有會有美軍的佔領，沒胡官僚主義的延...	1
4	作者在少年時即喜閱讀，能看出他精讀了無數經典，因而他有一個龐大的內心世界。他的作品最難能可貴...	1
...	...	...
80437	以前幾乎天天吃，現在調料什麼都不放，	0
80438	昨天訂涼皮兩份，什麼調料都沒有放，就放了點麻油，特別難吃，丟了一份，再也不想吃了	0
80439	涼皮太辣,吃不下都	0
80440	本來遲到了還自己點!!!	0
80441	肉夾饃不錯，羊肉泡饃醬肉包很一般。涼麵沒想象中好吃。送餐倒是很快。	0



後續步驟：[使用 df 生成程式碼](#) [查看建議的圖表](#) [New interactive sheet](#)

df.dtypes



	0
text	object
label	int64



Convert the format of y

Convert the format of y from int to LongTensor

Convert label using one-hot representation 輸出資料格式one-hot轉換

轉成用2個節點表達兩類

類別0: [1 0]

類別1: [0 1]

如果是3個類別用3個節點表之:

類別0: [1 0 0]

類別1: [0 1 0]

類別2: [0 0 1]

負面情緒Negative 0 --> [1 0]

正面情緒Positive 1 --> [0 1]

Easy Representation 0,1,2... (內部會自動轉換為one-hot)

負面情緒Negative 0 --> [0]

正面情緒Positive 1 --> [1]

如果是3個類別

中立情緒 Neutral 2 --> [2]

```
# Map labels to integers
categories=['負面','正面']
```

```
label_to_id = { cate : i for i, cate in enumerate(categories)}
```

```
label_to_id
```

```
{'負面': 0, '正面': 1}
```

```
id_to_label = { i : cate for i, cate in enumerate(categories)}
```

```
id_to_label
```

```
{0: '負面', 1: '正面'}
```

開始使用 AI 編寫或[生成](#)程式碼。

Sample some examples for demonstration

```
#df = df.sample(10000)
```

Conver pandas dataframe to Huggingface Dataset

```
dataset = datasets.Dataset.from_pandas(df, preserve_index=False)
# eval_data = Dataset.from_pandas(X_eval)
```

```
dataset
```

```
Dataset({
  features: ['text', 'label'],
  num_rows: 80442
})
```

```
dataset[0]
```

```
{'text': '做父母一定要有劉墉這樣的心態，不斷地學習，不斷地進步，不斷地給自己補充新鮮血液，讓自己保持一顆年輕的心。我想，這是他能很好的和孩子溝通的一個重要因素。讀劉墉的文章，總能讓我看到一個快樂的平易近人的父親，他始終站在和孩子同樣的高度，給孩子創造著一個充滿愛和自由的生活環境。很喜歡劉墉在字裡行間流露出的做父母的那種小狡黠，讓人總是忍俊不禁，父母和子女之間有時候也是一種戰鬥，武力爭鬥過於低級了，智力較量才更有趣味。所以，做父母
```

的得加把勁了，老思想老觀念注定會一敗塗地，生命不息，學習不止。家庭教育，真的是樂在其中。’，
'label': 1}

```
dataset.to_pandas().head(5)
```



	text	label
0	做父母一定要有劉墉這樣的心態，不斷地學習，不斷地進步，不斷地給自己補充新鮮血液，讓自己保持一...	1
1	作者真有英國人嚴謹的風格，提出觀點、進行論述論證，儘管本人對物理學瞭解不深，但是仍然能感受到...	1
2	作者長篇大論借用詳細報告數據處理工作和計算結果支持其新觀點。為什麼荷蘭曾經縣有歐洲最高的生產...	1
3	作者在戰幾時之前用了‘擁抱’令人叫絕。日本如果沒有戰敗，就有會有美軍的佔領，沒胡官僚主義的延...	1
4	作者在少年時即喜閱讀，能看出他精讀了無數經典，因而他有一個龐大的內心世界。他的作品最難能可貴	1



Load Tokenizer

```
# model_id = "google/gemma-3-1b"
# model_id = "Qwen/Qwen2.5-0.5B"
model_id = "Qwen/Qwen2.5-0.5B-instruct"
```

```
tokenizer = AutoTokenizer.from_pretrained(model_id)
```



```
/usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as :
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
```

```
warnings.warn(
```

```
tokenizer_config.json: 100% 7.30k/7.30k [00:00<00:00, 459kB/s]
```

```
vocab.json: 100% 2.78M/2.78M [00:00<00:00, 6.74MB/s]
```

```
merges.txt: 100% 1.67M/1.67M [00:00<00:00, 49.9MB/s]
```

```
tokenizer.json: 100% 7.03M/7.03M [00:00<00:00, 14.1MB/s]
```

```
# 必須在Huggingface註冊，取得API token才能下載模型
#access_token = "???"
#tokenizer = AutoTokenizer.from_pretrained(model_id, token=access_token)
```

Tokenize text

```
def tokenize_function(example):
    return tokenizer(
        example["text"],
        #padding="max_length",
        max_length=512,
        truncation=True,
    )
```

```
tokenized_dataset = dataset.map(tokenize_function, batched=True)
```



```
Map: 100% 80442/80442 [00:13<00:00, 6755.43 examples/s]
```

```
tokenized_dataset
```



```
Dataset({
  features: ['text', 'label', 'input_ids', 'attention_mask'],
  num_rows: 80442
})
```

```
# tokenized_dataset[0]
```

Split dataset for training and testing

✓ Split dataset: Train, Test (Val)

Training set: 訓練資料集 -->給模型讀進去訓練

Test set: 測試資料集 -->驗證或測試模型的準確度

Split dataset into 90% for training and 10% for testing

```
tokenized_dataset = tokenized_dataset.train_test_split(test_size=0.05, seed=1234)
train_data = tokenized_dataset["train"]
test_data = tokenized_dataset["test"]
```

```
train_data
```

```
Dataset({
  features: ['text', 'label', 'input_ids', 'attention_mask'],
  num_rows: 76419
})
```

```
print(test_data)
```

```
Dataset({
  features: ['text', 'label', 'input_ids', 'attention_mask'],
  num_rows: 4023
})
```

```
train_data[0]
```

```
{'text': '不滿意，色差太大！做工一般！！',
 'label': 0,
 'input_ids': [16530,
 101496,
 36589,
 3837,
 38035,
 99572,
 102791,
 6313,
 115238,
 100141,
 31251],
 'attention_mask': [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]}
```

✓ 定義Model

這裡的做法有簡單的版本也有複雜的版本

簡單版:

```
model = AutoModelForSequenceClassification.from_pretrained(
    model_id, # 模型名稱
    num_labels=len(categories), # Number of output labels
)
```

複雜版:

```
from transformers import Qwen2Model, Trainer, TrainingArguments, AutoTokenizer, AutoModel
from torch import nn
from transformers.modeling_outputs import SequenceClassifierOutput
import os
```

```
import torch
import os
import torch.nn.functional as F
from torch import nn
```

```
class QwenForClassifier(nn.Module):
    def __init__(self, base_model, hidden_size, num_labels):
        super(QwenForClassifier, self).__init__()
        # 凍結 base model 的參數
        self.base_model = base_model

        for param in self.base_model.parameters():
            param.requires_grad = False
```



```

# 注意力池化機制
self.attention_pooler = nn.Sequential(
    nn.Linear(hidden_size, 128),
    nn.Tanh(),
    nn.Linear(128, 1)
)

# 多層融合權重 (最後4層)
self.layer_weights = nn.Parameter(torch.ones(4) / 4)

# 增強型分類器
self.classifier = nn.Sequential(
    nn.Linear(hidden_size, 256),
    nn.LayerNorm(256),
    nn.GELU(),
    nn.Dropout(0.2),

    nn.Linear(256, 128),
    nn.LayerNorm(128),
    nn.GELU(),
    nn.Dropout(0.1),

    nn.Linear(128, num_labels)
)

# 保存配置
self.config = base_model.config
self.config.num_labels = num_labels

def forward(self, input_ids, attention_mask=None, labels=None):
    # 獲取所有隱藏層狀態
    outputs = self.base_model(
        input_ids=input_ids,
        attention_mask=attention_mask,
        output_hidden_states=True
    )

    # 獲取最後4層隱藏狀態
    hidden_states = outputs.hidden_states
    if hidden_states is None:
        # 如果模型沒有返回hidden_states, 使用last_hidden_state
        last_hidden = outputs.last_hidden_state
        sequence_output = last_hidden
    else:
        # 融合最後4層 (或可用層數)
        last_layers = hidden_states[-4:] if len(hidden_states) >= 4 else hidden_states[1:]
        layer_weights = F.softmax(self.layer_weights[:len(last_layers)], dim=0)

        # 加權融合多層特徵
        sequence_output = torch.zeros_like(last_layers[0])
        for i, layer in enumerate(last_layers):
            sequence_output += layer_weights[i].unsqueeze(-1).unsqueeze(-1) * layer

    # 注意力池化
    attention_scores = self.attention_pooler(sequence_output)
    attention_probs = F.softmax(attention_scores, dim=1)
    context_vector = torch.matmul(attention_probs.transpose(-1, -2), sequence_output).squeeze(1)

    # 也計算平均池化向量
    mean_pooled = torch.mean(sequence_output, dim=1)

    # 結合注意力池化和平均池化 (殘差連接)
    combined_repr = context_vector + mean_pooled

    # 分類預測
    logits = self.classifier(combined_repr)

    # 計算損失
    loss = None
    if labels is not None:
        loss_fct = nn.CrossEntropyLoss()
        loss = loss_fct(logits, labels)

    return {"loss": loss, "logits": logits}

def save_model(self, output_dir=None):
    """保存分類器權重和配置"""
    os.makedirs(output_dir, exist_ok=True)

    # 保存分類器權重
    classifier_path = os.path.join(output_dir, "classifier_weights.pt")

```

```

model_dict = {
    'classifier': self.classifier.state_dict(),
    'attention_pooler': self.attention_pooler.state_dict(),
    'layer_weights': self.layer_weights,
    'config': {
        'num_labels': self.config.num_labels,
        'hidden_size': self.config.hidden_size
    }
}

torch.save(model_dict, classifier_path)
print(f"已保存分類器權重至 {classifier_path}")

def load_model(self, model_dir, device=None):
    """載入分類器權重"""
    if device is None:
        device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

    classifier_path = os.path.join(model_dir, "classifier_weights.pt")
    if os.path.exists(classifier_path):
        model_dict = torch.load(classifier_path, map_location=device, weights_only=True)

        # 載入各組件
        self.classifier.load_state_dict(model_dict['classifier'])
        self.attention_pooler.load_state_dict(model_dict['attention_pooler'])
        self.layer_weights.data = model_dict['layer_weights'].to(device)

        print(f"已載入分類器權重: {classifier_path}")
        return True
    else:
        print(f"警告: 找不到分類器權重檔案 {classifier_path}")
        return False

```

✧ 初始化模型

分類任務：兩者都可用，但 `AutoModel` 更輕量

生成功能：只有 `AutoModelForCausalLM` 支持

內存使用：`AutoModelForCausalLM` 通常較大，因為包含了完整的語言模型頭

在 `Qwen2` 情感分類模型中，使用 `AutoModelForCausalLM` 更為靈活，因為它既可以進行分類，也保留了原始的文本生成功能。

```

# AutoModelForCausalLM 輸出
outputs = base_model(input_ids, attention_mask)
# 輸出包含：
# - loss: (可選) 語言模型損失
# - logits: 張量，形狀為 [batch_size, sequence_length, vocab_size]
# - past_key_values: (可選) 用於加速解碼的過去狀態
# - hidden_states: (可選) 所有隱藏層狀態的元組
# - attentions: (可選) 注意力權重的元組

# AutoModel 輸出
outputs = base_model(input_ids, attention_mask)
# 輸出包含：
# - last_hidden_state: 張量，形狀為 [batch_size, sequence_length, hidden_size]
# - hidden_states: (可選) 所有隱藏層狀態的元組
# - attentions: (可選) 注意力權重的元組

```

```
len(categories)
```

```
↔ 2
```

```

# 在外部先載入base_model預訓練權重(不包含分類層)
full_model = AutoModelForCausalLM.from_pretrained(model_id)
#
hidden_size = full_model.config.hidden_size
model = QwenForClassifier(full_model.model, hidden_size, num_labels= len(categories))

# 移動到指定設備
model = model.to(device)

```

```

config.json: 100% 659/659 [00:00<00:00, 54.1kB/s]

Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regular HTTP download. For better ;
WARNING:huggingface_hub.file_download:Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back
model.safetensors: 100% 988M/988M [00:05<00:00, 185MB/s]

Sliding Window Attention is enabled but not implemented for `sdpa`; unexpected results may be encountered.
generation_config.json: 100% 242/242 [00:00<00:00, 21.1kB/s]

```

full_model.model

```

Qwen2Model(
  (embed_tokens): Embedding(151936, 896)
  (layers): ModuleList(
    (0-23): 24 x Qwen2DecoderLayer(
      (self_attn): Qwen2Attention(
        (q_proj): Linear(in_features=896, out_features=896, bias=True)
        (k_proj): Linear(in_features=896, out_features=128, bias=True)
        (v_proj): Linear(in_features=896, out_features=128, bias=True)
        (o_proj): Linear(in_features=896, out_features=896, bias=False)
      )
      (mlp): Qwen2MLP(
        (gate_proj): Linear(in_features=896, out_features=4864, bias=False)
        (up_proj): Linear(in_features=896, out_features=4864, bias=False)
        (down_proj): Linear(in_features=4864, out_features=896, bias=False)
        (act_fn): SiLU()
      )
      (input_layernorm): Qwen2RMSNorm((896,), eps=1e-06)
      (post_attention_layernorm): Qwen2RMSNorm((896,), eps=1e-06)
    )
  )
  (norm): Qwen2RMSNorm((896,), eps=1e-06)
  (rotary_emb): Qwen2RotaryEmbedding()
)

```

full_model.base_model

```

Qwen2Model(
  (embed_tokens): Embedding(151936, 896)
  (layers): ModuleList(
    (0-23): 24 x Qwen2DecoderLayer(
      (self_attn): Qwen2Attention(
        (q_proj): Linear(in_features=896, out_features=896, bias=True)
        (k_proj): Linear(in_features=896, out_features=128, bias=True)
        (v_proj): Linear(in_features=896, out_features=128, bias=True)
        (o_proj): Linear(in_features=896, out_features=896, bias=False)
      )
      (mlp): Qwen2MLP(
        (gate_proj): Linear(in_features=896, out_features=4864, bias=False)
        (up_proj): Linear(in_features=896, out_features=4864, bias=False)
        (down_proj): Linear(in_features=4864, out_features=896, bias=False)
        (act_fn): SiLU()
      )
      (input_layernorm): Qwen2RMSNorm((896,), eps=1e-06)
      (post_attention_layernorm): Qwen2RMSNorm((896,), eps=1e-06)
    )
  )
  (norm): Qwen2RMSNorm((896,), eps=1e-06)
  (rotary_emb): Qwen2RotaryEmbedding()
)

```

✓ 看看base_model與model有何不同?

model內部有: 一個base_model+輸出分類層。它被拼接為分類器, 可以做兩個類別的分類

base_model仍舊是GPT自回歸模型

但是在記憶體, 兩者是共享同一份base_model權重, model有多一個分類器的輸出層的部分

full_model

```

Qwen2ForCausalLM(
  (model): Qwen2Model(
    (embed_tokens): Embedding(151936, 896)
    (layers): ModuleList(
      (0-23): 24 x Qwen2DecoderLayer(
        (self_attn): Qwen2Attention(
          (q_proj): Linear(in_features=896, out_features=896, bias=True)
          (k_proj): Linear(in_features=896, out_features=128, bias=True)
          (v_proj): Linear(in_features=896, out_features=128, bias=True)
          (o_proj): Linear(in_features=896, out_features=896, bias=False)
        )
        (mlp): Qwen2MLP(

```

```

        (gate_proj): Linear(in_features=896, out_features=4864, bias=False)
        (up_proj): Linear(in_features=896, out_features=4864, bias=False)
        (down_proj): Linear(in_features=4864, out_features=896, bias=False)
        (act_fn): SiLU()
    )
    (input_layernorm): Qwen2RMSNorm((896,), eps=1e-06)
    (post_attention_layernorm): Qwen2RMSNorm((896,), eps=1e-06)
)
)
(norm): Qwen2RMSNorm((896,), eps=1e-06)
(rotary_emb): Qwen2RotaryEmbedding()
)
(lm_head): Linear(in_features=896, out_features=151936, bias=False)
)

```

model

```

↔ QwenForClassifier(
  (base_model): Qwen2Model(
    (embed_tokens): Embedding(151936, 896)
    (layers): ModuleList(
      (0-23): 24 x Qwen2DecoderLayer(
        (self_attn): Qwen2Attention(
          (q_proj): Linear(in_features=896, out_features=896, bias=True)
          (k_proj): Linear(in_features=896, out_features=128, bias=True)
          (v_proj): Linear(in_features=896, out_features=128, bias=True)
          (o_proj): Linear(in_features=896, out_features=896, bias=False)
        )
        (mlp): Qwen2MLP(
          (gate_proj): Linear(in_features=896, out_features=4864, bias=False)
          (up_proj): Linear(in_features=896, out_features=4864, bias=False)
          (down_proj): Linear(in_features=4864, out_features=896, bias=False)
          (act_fn): SiLU()
        )
        (input_layernorm): Qwen2RMSNorm((896,), eps=1e-06)
        (post_attention_layernorm): Qwen2RMSNorm((896,), eps=1e-06)
      )
    )
    (norm): Qwen2RMSNorm((896,), eps=1e-06)
    (rotary_emb): Qwen2RotaryEmbedding()
  )
  (attention_pooler): Sequential(
    (0): Linear(in_features=896, out_features=128, bias=True)
    (1): Tanh()
    (2): Linear(in_features=128, out_features=1, bias=True)
  )
  (classifier): Sequential(
    (0): Linear(in_features=896, out_features=256, bias=True)
    (1): LayerNorm((256,), eps=1e-05, elementwise_affine=True)
    (2): GELU(approximate='none')
    (3): Dropout(p=0.2, inplace=False)
    (4): Linear(in_features=256, out_features=128, bias=True)
    (5): LayerNorm((128,), eps=1e-05, elementwise_affine=True)
    (6): GELU(approximate='none')
    (7): Dropout(p=0.1, inplace=False)
    (8): Linear(in_features=128, out_features=2, bias=True)
  )
)

```

這與model是一樣的模型??

model.base_model

```

↔ Qwen2Model(
  (embed_tokens): Embedding(151936, 896)
  (layers): ModuleList(
    (0-23): 24 x Qwen2DecoderLayer(
      (self_attn): Qwen2Attention(
        (q_proj): Linear(in_features=896, out_features=896, bias=True)
        (k_proj): Linear(in_features=896, out_features=128, bias=True)
        (v_proj): Linear(in_features=896, out_features=128, bias=True)
        (o_proj): Linear(in_features=896, out_features=896, bias=False)
      )
      (mlp): Qwen2MLP(
        (gate_proj): Linear(in_features=896, out_features=4864, bias=False)
        (up_proj): Linear(in_features=896, out_features=4864, bias=False)
        (down_proj): Linear(in_features=4864, out_features=896, bias=False)
        (act_fn): SiLU()
      )
      (input_layernorm): Qwen2RMSNorm((896,), eps=1e-06)
      (post_attention_layernorm): Qwen2RMSNorm((896,), eps=1e-06)
    )
  )
  (norm): Qwen2RMSNorm((896,), eps=1e-06)
  (rotary_emb): Qwen2RotaryEmbedding()
)

```

✓ 模型怎麼用?

尚未訓練的模型 (classifier的數據都是0)

```
# Function to make predictions
def predict_sentiment(text, model, tokenizer, device):
    # Tokenize the input text
    inputs = tokenizer(
        text,
        max_length=512,
        truncation=True,
        return_tensors="pt"
    ).to(device)

    # Get model predictions
    with torch.no_grad():
        outputs = model(**inputs)

    # Extract logits and apply softmax to get probabilities
    # logits = outputs.logits
    logits = outputs["logits"] # 取出 logits

    probabilities = torch.nn.functional.softmax(logits, dim=-1)

    # Get the predicted class (0: negative, 1: positive)
    predicted_class = torch.argmax(probabilities, dim=-1).item()

    # Get the class name using id_to_label
    predicted_label = id_to_label[predicted_class]

    # Get the confidence score
    confidence = probabilities[0][predicted_class].item()

    return {
        "text": text,
        "sentiment": predicted_label,
        "confidence": round(confidence, 2),
        "probabilities": {
            id_to_label[i]: round(prob.item(), 2) for i, prob in enumerate(probabilities[0])
        }
    }
```

```
text = "今天天氣真好，我很開心"
predict_sentiment(text, model, tokenizer, device)
```

```
↺ {'text': '今天天氣真好，我很開心',
   'sentiment': '正面',
   'confidence': 0.72,
   'probabilities': {'負面': 0.28, '正面': 0.72}}
```

```
text = "這個產品品質差，服務更糟糕"
predict_sentiment(text, model, tokenizer, device)
```

```
↺ {'text': '這個產品品質差，服務更糟糕',
   'sentiment': '正面',
   'confidence': 0.75,
   'probabilities': {'負面': 0.25, '正面': 0.75}}
```

```
text = "這家餐廳的食物美味，環境也很舒適"
predict_sentiment(text, model, tokenizer, device)
```

```
↺ {'text': '這家餐廳的食物美味，環境也很舒適',
   'sentiment': '正面',
   'confidence': 0.55,
   'probabilities': {'負面': 0.45, '正面': 0.55}}
```

✓ 模型每個參數層都是Trainable

```
def print_model_details(model):
    print("\n" + "="*80)
    print("MODEL ARCHITECTURE WITH TRAINABILITY STATUS")
    print("="*80)

    trainable_params = 0
```

```
non_trainable_params = 0

# Print each layer with details
for idx, (name, layer) in enumerate(model.named_modules(), 1):
    if not list(layer.named_children()): # Only print leaf nodes (actual layers)
        param_count = sum(p.numel() for p in layer.parameters())
        status = "TRAINABLE" if any(p.requires_grad for p in layer.parameters()) else "NON-TRAINABLE"

        print(f"\nLayer #{idx:02d}")
        print(f"├─ Name: {name}")
        print(f"├─ Type: {layer.__class__.__name__}")
        print(f"├─ Details: {layer}")
        print(f"├─ Parameters: {param_count:,}")
        print(f"└─ Status: {status}")

        if status == "TRAINABLE":
            trainable_params += param_count
        else:
            non_trainable_params += param_count

total_params = trainable_params + non_trainable_params
trainable_percentage = (trainable_params / total_params) * 100 if total_params > 0 else 0

print("\n" + "="*80)
print(f"Total Trainable Parameters: {trainable_params:,}")
print(f"Total Non-Trainable Parameters: {non_trainable_params:,}")
print(f"Total Parameters: {total_params:,}")
print(f"Trainable Parameters Percentage: {trainable_percentage:.2f}%")
print("="*80)

print_model_details(model)
```



```
total non-trainable parameters: 494,032,768
Total Parameters: 494,411,267
Trainable Parameters Percentage: 0.08%
```

```
# 檢查原始參數是否可訓練
# print("Classifier weight requires_grad:", model.classifier.weight.requires_grad)
```

✓ 模型主體固定參數，只有極少數參數是Trainable

```
#model.print_trainable_parameters()
```

```
def print_trainable_parameters(model):
    trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
    all_params = sum(p.numel() for p in model.parameters())
    print(f"trainable params: {trainable_params:,d} || all params: {all_params:,d} || trainable%: {100 * trainable_params / all_params:.08%}")
```

```
print_trainable_parameters(model)
```

```
↻ trainable params: 378,503 || all params: 494,411,271 || trainable%: 0.08%
```

```
print_model_details(model)
```

```
↻ | Parameters: 512
  | Status: TRAINABLE
```

```
Layer #326
| Name: classifier.2
| Type: GELU
| Details: GELU(approximate='none')
| Parameters: 0
| Status: NON-TRAINABLE
```

```
Layer #327
| Name: classifier.3
| Type: Dropout
| Details: Dropout(p=0.2, inplace=False)
| Parameters: 0
| Status: NON-TRAINABLE
```

```
Layer #328
| Name: classifier.4
| Type: Linear
| Details: Linear(in_features=256, out_features=128, bias=True)
| Parameters: 32,896
| Status: TRAINABLE
```

```
Layer #329
| Name: classifier.5
| Type: LayerNorm
| Details: LayerNorm((128,), eps=1e-05, elementwise_affine=True)
| Parameters: 256
| Status: TRAINABLE
```

```
Layer #330
| Name: classifier.6
| Type: GELU
| Details: GELU(approximate='none')
| Parameters: 0
| Status: NON-TRAINABLE
```

```
Layer #331
| Name: classifier.7
| Type: Dropout
| Details: Dropout(p=0.1, inplace=False)
| Parameters: 0
| Status: NON-TRAINABLE
```

```
Layer #332
| Name: classifier.8
| Type: Linear
| Details: Linear(in_features=128, out_features=2, bias=True)
| Parameters: 258
| Status: TRAINABLE
```

```
=====
Total Trainable Parameters: 378,499
Total Non-Trainable Parameters: 494,032,768
Total Parameters: 494,411,267
Trainable Parameters Percentage: 0.08%
=====
```

✓ Use EOS token as padding

```
tokenizer.pad_token
```



```
tokenizer.eos_token
```



```
model.pad_token_id = tokenizer.eos_token_id # Use EOS token as padding
```

✓ Train

```
class CustomTrainer(Trainer):
    def save_model(self, output_dir=None, _internal_call=False):
        """保存分類器權重和配置"""
        # 使用預設output_dir如果未指定
        output_dir = output_dir if output_dir is not None else self.args.output_dir
        os.makedirs(output_dir, exist_ok=True)

        # 保存分類器權重
        classifier_path = os.path.join(output_dir, "classifier_weights.pt")
        model_dict = {
            'classifier': self.model.classifier.state_dict(),
            'attention_pooler': self.model.attention_pooler.state_dict(),
            'layer_weights': self.model.layer_weights,
            'config': {
                'num_labels': self.model.config.num_labels,
                'hidden_size': self.model.config.hidden_size
            }
        }

        # Save model config (required by HF Trainer)
        if hasattr(self.model, "config"):
            self.model.config.save_pretrained(output_dir)

        torch.save(model_dict, classifier_path)
        print(f"已保存分類器權重至 {classifier_path}")

        # 保存訓練狀態
        super().save_state()

        return output_dir

#metric = evaluate.combine(["accuracy", "f1", "precision", "recall"])
#metric = evaluate.combine(["accuracy"])
def compute_metrics(eval_pred):
    predictions, labels = eval_pred
    predictions = np.argmax(predictions, axis=1) # Convert probabilities to predicted labels
    metric = evaluate.load('accuracy')
    return metric.compute(predictions=predictions, references=labels)

# requires several GB of GPU memory
training_args = TrainingArguments(
    output_dir="checkpoints_v4", # Output directory for checkpoints
    learning_rate=5e-5, # Learning rate for the optimizer
    weight_decay=0.01, # Weight decay for regularization
    warmup_steps=500,
    seed=101,

    per_device_train_batch_size=26, # Batch size per device
    per_device_eval_batch_size=3, # Batch size per device for evaluation

    num_train_epochs=5, # Number of training epochs

    eval_strategy='steps', # Evaluate after each epoch
    save_strategy="steps", # Save model checkpoints after each epoch

    load_best_model_at_end=True, # Load the best model based on the chosen metric
    save_total_limit=2,
    push_to_hub=False, # Disable pushing the model to the Hugging Face Hub
```



```
report_to="none", # Disable logging to Weight&Bias
#fp16=True, # 是否用此精度訓練Whether to use fp16 16-bit (mixed) precision training instead of 32-bit training.
#bf16=True, # for 新型GPU 才能設定bf16
logging_steps=1000,
)

trainer = CustomTrainer(
    model=model, # The LoRA-adapted model
    #tokenizer=tokenizer, # Tokenizer for the model
    processing_class=tokenizer, # Tokenizer for the model

    train_dataset=train_data, # Training dataset
    eval_dataset=test_data, # Evaluation dataset
    args=training_args, # Training arguments
    compute_metrics=compute_metrics, # Function to calculate evaluation metrics
)
```

✦ Resume training from the last checkpoint if available

接續訓練

訓練後，手動載入之前訓練的分類器權重

```
# checkpoint_path = "./checkpoints_v3/checkpoint-3167"
# model_path = "trained_classifier_v3"
# model.load_model(checkpoint_path)
```

✦ Let's train the model

```
%%time
model.config.use_cache = False # Disable cache for faster training
trainer.train()
# trainer.train(resume_from_checkpoint=True) # Resume training from a checkpoint
#trainer.train(resume_from_checkpoint="./checkpoints_v1/checkpoint-19002")
```



Step	Training Loss	Validation Loss	Accuracy
1000	0.425100	0.242156	0.911012
2000	0.246600	0.242669	0.911509
3000	0.239900	0.219461	0.922197
4000	0.220200	0.227337	0.922197
5000	0.221700	0.229094	0.920457
6000	0.212500	0.208000	0.926174
7000	0.205500	0.200682	0.928909
8000	0.201600	0.205975	0.927666
9000	0.199000	0.199133	0.927914



已保存分類器權重至 checkpoints_v4/checkpoint-500/classifier_weights.pt

Downloading builder script: 100% 4.20k/4.20k [00:00<00:00, 384kB/s]

已保存分類器權重至 checkpoints_v4/checkpoint-1000/classifier_weights.pt
已保存分類器權重至 checkpoints_v4/checkpoint-1500/classifier_weights.pt
已保存分類器權重至 checkpoints_v4/checkpoint-2000/classifier_weights.pt
已保存分類器權重至 checkpoints_v4/checkpoint-2500/classifier_weights.pt
已保存分類器權重至 checkpoints_v4/checkpoint-3000/classifier_weights.pt
已保存分類器權重至 checkpoints_v4/checkpoint-3500/classifier_weights.pt
已保存分類器權重至 checkpoints_v4/checkpoint-4000/classifier_weights.pt
已保存分類器權重至 checkpoints_v4/checkpoint-4500/classifier_weights.pt
已保存分類器權重至 checkpoints_v4/checkpoint-5000/classifier_weights.pt
已保存分類器權重至 checkpoints_v4/checkpoint-5500/classifier_weights.pt
已保存分類器權重至 checkpoints_v4/checkpoint-6000/classifier_weights.pt
已保存分類器權重至 checkpoints_v4/checkpoint-6500/classifier_weights.pt
已保存分類器權重至 checkpoints_v4/checkpoint-7000/classifier_weights.pt
已保存分類器權重至 checkpoints_v4/checkpoint-7500/classifier_weights.pt
已保存分類器權重至 checkpoints_v4/checkpoint-8000/classifier_weights.pt
已保存分類器權重至 checkpoints_v4/checkpoint-8500/classifier_weights.pt
已保存分類器權重至 checkpoints_v4/checkpoint-9000/classifier_weights.pt
已保存分類器權重至 checkpoints_v4/checkpoint-9500/classifier_weights.pt

開始使用 AI 編寫或生成程式碼。

✓ Make a test

```
text = "今天天氣真好，我很開心"
predict_sentiment(text, model, tokenizer, device)
```

```
text = "這個產品品質差，服務更糟糕"
predict_sentiment(text, model, tokenizer, device)
```

✓ Save Model

```
#存lora_model
# model_path = "trained_lora_model_v1"
# trainer.model.save_pretrained(model_path)
```

```
# 這樣存檔 - 只會儲存分類器權重和設定
# model_path = "trained_classifier_v4"
# trainer.save_model(model_path)
# model.save_pretrained(model_path)
```

```
# 這樣存檔 - 只會儲存分類器權重和設定
model_path = "trained_classifier_v4"
model.save_model(model_path)
# model.save_pretrained(model_path)
```

✓ Evaluate the model

```
def predict(input_text):
    inputs = tokenizer(input_text, return_tensors="pt").to(device) # Convert to PyTorch tensors and move to GPU (if available)
    with torch.no_grad():
        #outputs = model(**inputs).logits # Get the model's output logits
        outputs = model(**inputs)['logits'] # Get the model's output logits
        y_prob = torch.sigmoid(outputs).tolist()[0] # Apply sigmoid activation and convert to list
    return np.round(y_prob, 2) # Round the predicted probability to 2 decimal places
```

```
def inference(text):
    inputs = tokenizer(text, return_tensors="pt", truncation=True, max_length=512).to(device)
    with torch.no_grad():
        outputs = model(**inputs)
    probs = torch.softmax(outputs['logits'], dim=1)
    #probs = torch.softmax(outputs.logits, dim=1)
    return probs.argmax().item()
```

```
df_test = pd.DataFrame(data=test_data)
```

```
df_test
```

```
input_text='''立法院今天發布人事通報，總務處長周傑調任參事室參事並代理總務處長，引發外界聯想立法院長韓國瑜出手進行人事調整。周傑今天受訪表示，正常調動部分媒體報導，質疑立法院總務處長人事異動，是否與先前立法院立委辦公室傳出有裝潢費昂貴、不合理等情形，或是日前立法院司法及法制委員會有委員提議要更改格局'''
predict(input_text)
```

```
input_text = "我不高興"
inference(input_text)
```

```
%%time
df_test['prediction'] = df_test['text'].map(predict)
df_test['y_pred'] = df_test['prediction'].apply(lambda x: np.argmax(x, axis=0))
```

```
#
# df_test['prediction'] = df_test['text'].map(inference)
```

開始使用 AI 編寫或生成程式碼。

```
accuracy = (df_test['y_pred'] == df_test['label']).mean()
print(f'Model Accuracy on Test Data: {accuracy:.4f}')
```

```
df_test.head()
```

開始使用 AI 編寫或生成程式碼。

```
cm = metrics.confusion_matrix(df_test['label'], df_test['y_pred'])
cm

print(metrics.classification_report(df_test['label'], df_test['y_pred']))
```

開始使用 AI 編寫或生成程式碼。

```
metrics.precision_score(df_test['label'], df_test['y_pred'], average='micro')

metrics.precision_score(df_test['label'], df_test['y_pred'], average='macro')

metrics.recall_score(df_test['label'], df_test['y_pred'], average='micro')

metrics.recall_score(df_test['label'], df_test['y_pred'], average='macro')
```

開始使用 AI 編寫或生成程式碼。

開始使用 AI 編寫或生成程式碼。

✓ Load model

```
# 在外部先載入base_model預訓練權重(不包含分類層)
# full_model = AutoModelForCausalLM.from_pretrained(model_id)
#
model_path = "trained_classifier_v4"
# model_path = "checkpoints_v3/checkpoint-4145"
hidden_size = full_model.config.hidden_size
model = QwenForClassifier(full_model.model, hidden_size, num_labels= len(categories))

model.load_model(model_path, device=device)

# 移動到指定設備
model = model.to(device)

text = "這個產品品質差，服務更糟糕"
predict_sentiment(text, model, tokenizer, device)

full_model
```

✓ Text generation fro full_model

```
from IPython.display import Markdown

def generate_text(input_prompt):

    messages = [
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": input_prompt}
    ]
    text = tokenizer.apply_chat_template(
        messages,
        tokenize=False,
        add_generation_prompt=True
    )
    model_inputs = tokenizer([text], return_tensors="pt").to(device)

    prompt_length = model_inputs['input_ids'].shape[1]

    generated_ids = full_model.generate(
        model_inputs.input_ids,
        max_new_tokens=512,
        pad_token_id=tokenizer.eos_token_id
    )
    response = tokenizer.decode(generated_ids[0][prompt_length:], skip_special_tokens=True)
```

```
return response
```

```
text="給出三個保持健康的提示。"  
result = generate_text(text)  
Markdown(result)
```

```
%%time  
text="我們如何減少空氣污染? 請給幾項重要的建議。"  
result = generate_text(text)  
Markdown(result)
```

開始使用 AI 編寫或生成程式碼。